

L12: Trigger

RECAP from Lab #11

- If

```
IF credit > 50000 THEN
    SET pCustomerLevel = 'PLATINUM';
ELSEIF credit <= 50000 AND credit > 10000 THEN
    SET pCustomerLevel = 'GOLD';
ELSE
    SET pCustomerLevel = 'SILVER';
END IF;
```

- Case

```
CASE customerCountry
    WHEN 'USA' THEN
        SET pShipping = '2-day Shipping';
    WHEN 'Canada' THEN
        SET pShipping = '3-day Shipping';
    ELSE
        SET pShipping = '5-day Shipping';
END CASE;
```

- Loop

```
insert_date: REPEAT
    INSERT INTO calendars(date,
        month, quarter, year)
        VALUES(currentDate,
            MONTH(currentDate),
            QUARTER(currentDate),
            YEAR(currentDate));
    SET currentDate =
        DATE_ADD(currentDate, INTERVAL 1
            DAY);
UNTIL currentDate > endDate
END REPEAT;
```

12.1 Trigger

A trigger is a stored program **invoked automatically** in response to an event such as insert, update, or delete that occurs in the associated table.

- A row-level trigger is activated for each row that is inserted, updated, or deleted. For example, if a table has 100 rows inserted, updated, or deleted, the trigger is automatically invoked 100 times for the 100 rows affected.
- A statement-level trigger is executed once for each transaction regardless of how many rows are inserted, updated, or deleted.

Trigger

There are three main parts:

- Event : is used to activate the trigger.
Before / After ...
- Condition : check whether the trigger should run.
[optional] SQL Expression/Query
- Action : consequence after the trigger run.
SQL statements

12.2 Create Trigger

Syntax

```
CREATE TRIGGER trigger_name  
{BEFORE | AFTER} {INSERT | UPDATE | DELETE}  
ON table_name FOR EACH ROW  
BEGIN  
    statements;  
END;
```

Syntax: Show Trigger

```
SHOW TRIGGERS;
```

Syntax: Drop Trigger

```
DROP TRIGGER trigger_name;
```

Create Trigger

Example

```
DELIMITER //
```

```
CREATE TRIGGER update_items_trigger
```

```
AFTER UPDATE
```

```
ON items
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    INSERT INTO item_changes (item_id, change_type)
```

```
    VALUES (NEW.id, 'UPDATE');
```

```
END;
```

```
//
```

```
DELIMITER ;
```

Human UPDATE

id	name	price
1	Item	50.00
2	B	60.00
3	Cat	30.00

UPDATE

id	name	price
1	Item	60.00
2	B	60.00
3	Cat	30.00



Trigger INSERT

change_id	item_id	change_type	change_timestamp
NULL	NULL	NULL	NULL

change_id	item_id	change_type	change_timestamp
1	1	UPDATE	2024-12-24 14:23:33

12.3 Trigger Modifiers

To distinguish between the value of the columns BEFORE and AFTER the event has fired, you use the **NEW** and **OLD** modifiers.

Trigger Event	OLD	NEW
INSERT		✓
UPDATE	✓	✓
DELETE	✓	

12.4 Create Multiple Triggers

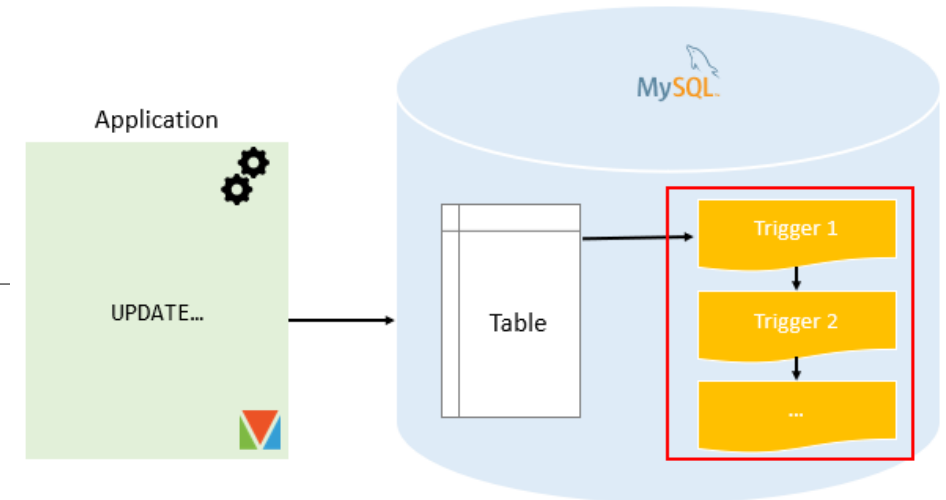
Syntax

```
DELIMITER $$

CREATE TRIGGER trigger_name
{BEFORE | AFTER} {INSERT | UPDATE | DELETE}
ON table_name FOR EACH ROW
{FOLLOWS | PRECEDES} existing_trigger_name
BEGIN
    statements
END$$

DELIMITER ;
```

- **FOLLOWS:** new trigger after existing trigger.
- **PRECEDES:** new trigger before existing trigger.



12.5 Trigger limitations

When you use a statement that does not use INSERT, DELETE or UPDATE statement to change data in a table, the triggers associated with the table are not invoked.

MySQL triggers cannot:

- Use SHOW, LOAD DATA, LOAD TABLE, BACKUP DATABASE, RESTORE, FLUSH and RETURN statements.
- Use prepared statements such as PREPARE and EXECUTE.
- Use statements that commit or rollback implicitly or explicitly such as COMMIT , ROLLBACK , START TRANSACTION , LOCK/UNLOCK TABLES , ALTER , CREATE , DROP , RENAME.